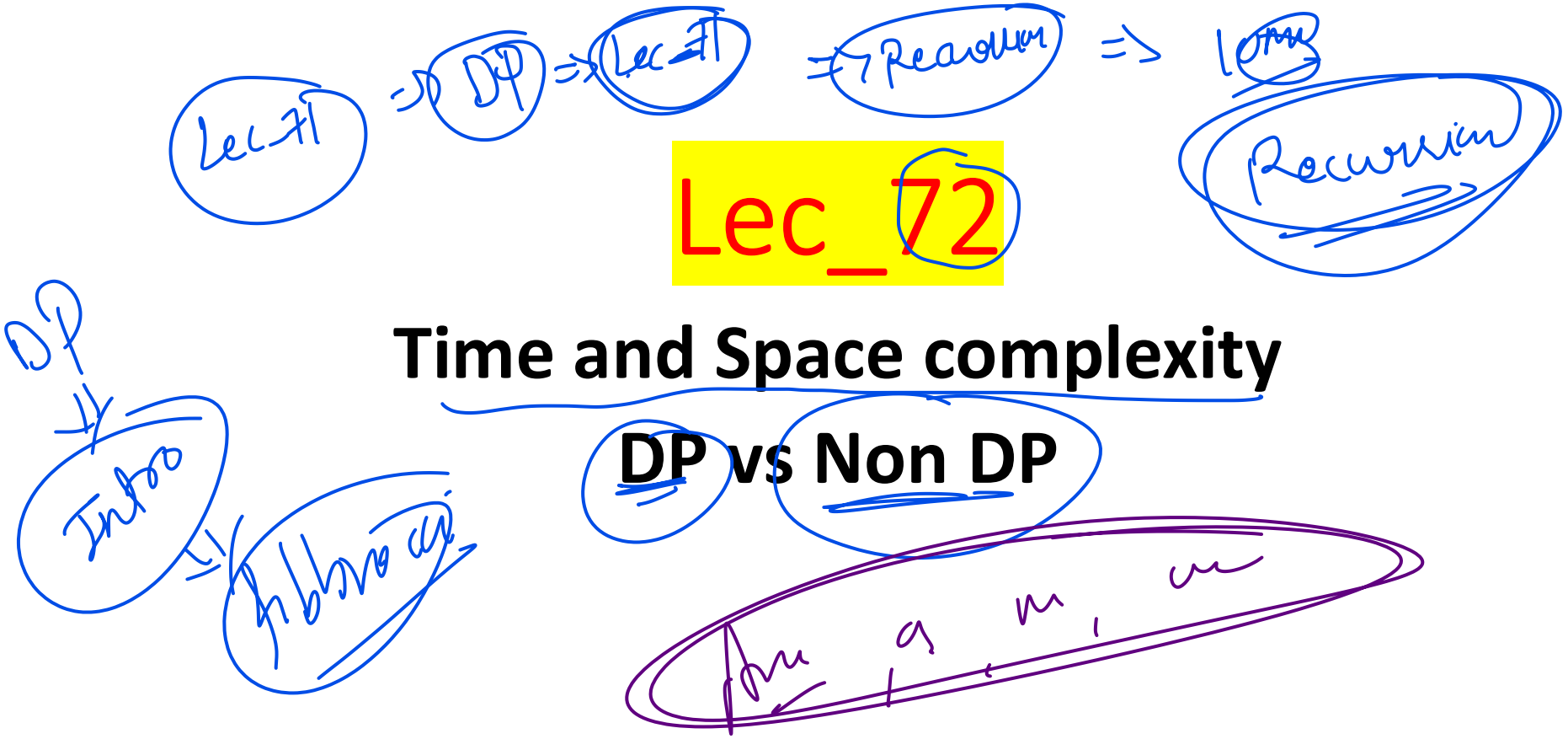

Lec_72

Time and Space complexity

DP vs Non DP

Without DP:

```
fib(n)
{ if(n==0||n==1)
    return n;
  a = fib(n-1)
  b = fib(n-2)
  return a+b;
}
```

0 1 2 3 5 8

Without DP in worst case 2^n fn calls and for each fn TC = O(1)

$2^n \times O(1) = 2^n$

SC, TC, DP V/S non DP

without DP: $T(n) = T(n-1) + T(n-2) + 1$

TC → $O(2^n)$

SC = O(n)

- to implement recursion stack Units are needed

- calculated thing need not to calculate again

$T[n+1] = \{-1\}$, $T[0] = -1$, $T(1) = -1$

$T(n) = -1$

With DP

```
fib(n)
{ if(n==0||n==1)
  { T[n] = n;
    return T[n];
  }
  if(T[n-1] == -1)
    T[n-1] = fib(n-1)
  if(T[n-2] == -1)
    T[n-2] = fib(n-2)
  T[n] = T[n-1] + T[n-2]
  return T[n]
}
```

With DP

fib(n) ⇒ fib(0) fib(1) fib(2) ... fib(n)

In worst case n+1 fn calls and TC of each fn call O(1)

TC = (n+1) O(1)

TC = O(n)

SC = O(n) + O(n+1)

stack Table

SC = O(2n+1)

SC = O(n)

for fibbo

non DP	DP
1. TC = $O(2^n)$	TC = $O(n)$
2. SC = $O(n)$	SC = $O(n)$
3. How many fn calls in worst case = 2^n	How many fn calls in worst case = $n+1$

DP:- In normal fibbnocci series same function we are calculating again and again thats why TC = $O(2^n)$, but why we are using DP we are not doing this, whenever it is calculated first time we will store it for further purpose and this called DP